

Head-Detector — Pipeline A

YOLOv9 head-pose detector with Norfair tracking and TimescaleDB logging.

What it does

- Detects people in a configurable ROI polygon using a YOLOv9 ONNX model
- Tracks individuals across frames with Norfair (IoU-based)
- Skips brand-new tracks (`obj.age == 0`) to suppress false detections
- Accumulates track lifetime data; **inserts one DB row per track at track death**
 - `timestamp` = actual entry time (first frame), not insert time
 - `dwelt_seconds` = full time in ROI
 - Tracks shorter than `min_elapsed_time` (`config2.yml`) are discarded
- Writes periodic `queue_state_snapshots` (queue count, avg/max dwell, lanes)
- Stores `gender='unknown'`, `age=NULL` (no face analytics in this pipeline)

Setup

```
cd Head-Detector
python -m venv venv
source venv/bin/activate          # Windows: venv\Scripts\activate
pip install -r requirements.txt
```

ROI configuration

Run once to define the queue zone polygon:

```
python pick_zone.py
```

Left-click to add points, Z to undo, Enter/Space to print the points: block, paste into `config.yml`.

Run

```
python main.py
python main.py --execution_provider cuda
python main.py --execution_provider tensorrt --inference_type fp16
python main.py --source path/to/video.mp4 --view-img
```

Configuration files

File	Key settings
<code>config.yml</code>	<code>camID</code> , <code>ip_address</code> , RTSP credentials, ONNX model path, ROI polygon
<code>config2.yml</code>	<code>max_age</code> , <code>max_distance_between_points</code> , <code>snapshot_interval</code> , <code>min_elapsed_time</code> , <code>debug_mode</code>

Camera ID note

Use camID: SIM_live_test_video when running against a local test video file so entries are excluded from real-data training queries.

Database tables written

Table	Written when
entrance_events	At track death (one row per confirmed person)
queue_state_snapshots	Every snapshot_interval seconds